

IQR Method for Outlier Detection

Concept: An outlier is a value that is unusually far from the rest of the data. The **IQR (Interquartile Range) method** is mainly used for detecting outliers in **skewed/non-normal data**.

1. Calculate IQR

$$\text{IQR} = Q3 - Q1$$

- **Q1** → 25th percentile
- **Q2** → Median (50th percentile)
- **Q3** → 75th percentile

2. Calculate Limits

$$\text{Lower Limit} = Q1 - 1.5 \times \text{IQR}$$

$$\text{Upper Limit} = Q3 + 1.5 \times \text{IQR}$$

Any value below the lower limit or above the upper limit is considered an **outlier**.

3. Python Example

```
Q1 = df['Marks'].quantile(0.25)
```

```
Q3 = df['Marks'].quantile(0.75)
```

$$\text{IQR} = Q3 - Q1$$

$$\text{lower} = Q1 - 1.5 * \text{IQR}$$

$$\text{upper} = Q3 + 1.5 * \text{IQR}$$

```
outliers = df[(df['Marks'] < lower) | (df['Marks'] > upper)]
```

4. Handling Outliers

Trimming: Remove rows containing outliers.

```
df = df[(df['Marks'] >= lower) & (df['Marks'] <= upper)]
```

Capping: Replace extreme values with the calculated limits.

```
df['Marks'] = df['Marks'].clip(lower, upper)
```

Key Idea:

Q1 & Q3 → Calculate IQR → Find Lower/Upper Limits → Detect Outliers → Trim or Cap them.

Percentile Method for Outlier Detection

Concept: Percentiles show the position of a value in a dataset. For example, the **99th percentile** means 99% of values are below it. The **50th percentile** is the median.

1. Define Limits

Commonly, values below the **1st percentile** or above the **99th percentile** are treated as outliers.

```
lower = df['Weight'].quantile(0.01)
```

```
upper = df['Weight'].quantile(0.99)
```

2. Trimming

Remove values outside the percentile limits.

```
df = df[(df['Weight'] >= lower) & (df['Weight'] <= upper)]
```

3. Winsorization

Instead of removing outliers, replace them with the percentile limits.

```
import numpy as np
```

```
df['Weight'] = np.where(
    df['Weight'] > upper, upper,
    np.where(df['Weight'] < lower, lower, df['Weight'])
)
```

Key Idea:

Calculate percentiles → Set lower/upper limits → Detect outliers → Trim or Winsorize.

When to use: Useful when you want a simple outlier-handling method, espec

Feature Construction & Feature Splitting

1. Feature Construction

Feature construction means **creating a new feature from existing features** using domain knowledge.

Example:

```
df['Family_Size'] = df['SibSp'] + df['Parch'] + 1
```

Here, Family_Size combines SibSp and Parch to give the total family size.

Purpose: Create more meaningful information that helps the ML model learn better patterns.

2. Feature Splitting

Feature splitting means **breaking one complex column into multiple useful features**.

Example:

```
df['Title'] = df['Name'].str.extract(r'\s*([^\s]*)\s*')
```

If Name contains:

"Sharma, Mr. Rahul"

It can extract:

Title → Mr

You can also split other information:

```
df['First_Name'] = df['Name'].str.split(',').str[0]
```

Purpose: Convert complex information into separate features that the model can understand more easily.

PCA — Principal Component Analysis

Concept: PCA is an **unsupervised dimensionality reduction technique** used for **feature extraction**. It converts many original features into a smaller number of new features called **Principal Components**, while trying to preserve maximum information.

Why PCA?

- Reduces the number of features.
- Makes ML models faster.
- Helps reduce complexity.
- Allows high-dimensional data to be visualized in **2D or 3D**.

Feature Selection vs Feature Extraction:

- **Feature Selection:** Selects important existing columns and removes others.
- **Feature Extraction:** Creates new features from existing features. PCA uses feature extraction.

How PCA works: PCA finds directions in the data where the **variance is maximum**. The first principal component (**PC1**) captures the maximum variance, the second (**PC2**) captures the next highest variance while being perpendicular to PC1, and so on.

Example:

PCA — Mathematical Formulation

Goal: PCA reduces dimensions while preserving maximum possible **variance (information)**.

1. Mean Centering

Subtract the mean from each feature:

```
X_centered = X - X.mean(axis=0)
```

2. Covariance Matrix

The covariance matrix shows the **variance of individual features** and their relationships.

```
cov_matrix = np.cov(X_centered, rowvar=False)
```

- Diagonal → variance
- Off-diagonal → covariance between features

3. Eigenvalues & Eigenvectors

Calculate eigenvalues and eigenvectors of the covariance matrix:

```
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
```

- **Eigenvectors** → directions of principal components
- **Eigenvalues** → amount of variance captured by each component

Sort eigenvalues in descending order and select the corresponding top eigenvectors.

4. Projection

Project the centered data onto the selected principal components:

```
X_pca = X_centered @ selected_eigenvectors
```

For example, if original data has **3 features** and you select 2 components:

3D data → PCA → 2D data

Key Idea

Mean Centering → Covariance Matrix → Eigenvalues/Eigenvectors → Select Top Components → Project Data

The component with the **largest eigenvalue captures the maximum variance**, so it becomes **PC1**. The next becomes **PC2**, and so on.

PCA — Practical Implementation with MNIST

Concept: MNIST contains handwritten digit images of size **28×28 pixels**, so each image has **784 features**. PCA reduces these 784 features into fewer **principal components** while preserving most of the important information. This reduces computation time and can make models like **KNN** faster.

1. Standardization

Before applying PCA, scale the features so that differences in feature magnitude don't affect PCA.

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

2. Apply PCA

For example, reduce **784 features to 100 components**:

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=100)
```

```
X_pca = pca.fit_transform(X_scaled)
```

```
print(X_pca.shape)
```

784 features → 100 principal components

The new 100 features are combinations of the original 784 pixels.

3. Explained Variance

Explained variance ratio tells us how much information/variance each principal component preserves.

```
pca.explained_variance_ratio_
```

To calculate the total variance captured:

```
import numpy as np
```

```
cumulative_variance = np.cumsum(  
    pca.explained_variance_ratio_  
)
```

```
print(cumulative_variance)
```

For example:

PC1 → 15%

PC2 → 10%

PC3 → 8%

...

The cumulative value tells us how much total information is retained by multiple components.

4. Automatically Select Components

Instead of manually choosing 100 components, we can ask PCA to preserve **95% of the variance**:

```
pca = PCA(n_components=0.95)
```

```
X_pca = pca.fit_transform(X_scaled)
```

```
print(pca.n_components_)
```

Scikit-learn automatically finds the number of components required to retain approximately **95% of the variance**.

5. PCA for 2D Visualization

PCA can reduce the data to just **2 dimensions**, allowing us to visualize the MNIST digits.

```
pca = PCA(n_components=2)
```

```
X_2d = pca.fit_transform(X_scaled)
```

Now:

784 dimensions → 2 dimensions

The different digits can then be plotted to see whether they form separate clusters.

6. Scree / Cumulative Variance Plot

```
import matplotlib.pyplot as plt
```

```
plt.plot(np.cumsum(pca.explained_variance_ratio_))
```

```
plt.xlabel('Number of Components')
```

```
plt.ylabel('Cumulative Explained Variance')
```

```
plt.show()
```

This helps determine how many components are needed to retain a desired amount of information.

Why PCA is Useful

Without PCA:

MNIST → 784 features → KNN

With PCA:

MNIST → 784 features → PCA → 100 features → KNN

Fewer features generally mean **less computation, faster training/prediction, reduced complexity, and easier visualization.**

Limitations of PCA

PCA may not work well when:

- Important information is distributed across many dimensions.
 - The data contains **non-linear relationships**.
 - Classes are not well separated using linear projections.
-

Simple Linear Regression

Concept: Linear Regression is a **supervised learning algorithm** used for **regression problems**, where the output is a continuous numerical value. It tries to find the **best-fit straight line** between input and output variables.

Example: Predicting a student's **package** based on their **CGPA**.

Types

- **Simple Linear Regression:** One input feature.
- **Multiple Linear Regression:** Multiple input features.

- **Polynomial Regression:** Uses polynomial features to model non-linear relationships.

Mathematical Formula

The equation of a simple linear regression line is:

$$y = mx + b$$

- $y \rightarrow$ predicted output
- $x \rightarrow$ input feature
- $m \rightarrow$ slope/weight
- $b \rightarrow$ intercept

The model tries to find the values of m and b that produce the **minimum prediction error**.

Example

CGPA $\rightarrow$ Package

7.0 $\rightarrow$ 4 LPA

7.5 $\rightarrow$ 5 LPA

8.0 $\rightarrow$ 6 LPA

8.5 $\rightarrow$ 7 LPA

The model learns the relationship and can predict the package for a new CGPA.

Implementation

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

df = pd.read_csv("placement.csv")

X = df[['CGPA']]
y = df['Package']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

Important Methods

```
model.fit(X_train, y_train)
```

fit() → learns the relationship between input and output.

```
model.predict(X_test)
```

predict() → predicts output for new/unseen data.

```
model.coef_
```

coef_ → gives the slope/weight.

```
model.intercept_
```

intercept_ → gives the y-intercept.

Key Idea

Input → Learn best-fit line → Minimize error → Predict continuous output

Example:

CGPA → Linear Regression → Predicted Package

This video provides an end-to-end guide to **Simple Linear Regression**, covering both the mathematical derivation and a practical implementation from scratch in Python.

Mathematical Formulation (0:00 - 40:06)

- **Goal:** The objective is to find a line represented by the equation that best fits the data by minimizing the error between actual points and the line (the "Best Fit Line").
- **Error Minimization:** The video uses the **Ordinary Least Squares (OLS)** method. The error function (or loss function) is defined as the sum of squared residuals: (13:18).
- **Deriving 'm' and 'b':** By taking partial derivatives of the error function with respect to m and b and setting them to zero, the following formulas are derived:
 - **Slope (m):** (39:10)
 - **Intercept (b):** (35:23)
- **Solutions:** The video distinguishes between **Closed-form solutions** (direct formulas like OLS) and **Non-closed-form solutions** (like Gradient Descent, used for higher dimensions) (3:12).

Coding from Scratch (40:06 - 53:10)

- **Implementation:** The author creates a custom Python class called LinearRegression (40:31).

- **Methods:**
 - `fit()`: Calculates the mean of `X` and `y`, computes the numerator and denominator for `coef`, determines `intercept`, and stores them.
 - `predict()`: Returns the predicted value using the derived `coef` and `intercept` for new inputs (50:35).
 - **Validation:** The custom class is tested against the dataset used in the previous intuition video, yielding identical results to the standard scikit-learn implementation (49:40)
-

Regression Evaluation Metrics

Regression metrics tell us **how well a model's predicted values match the actual values**.

Example:

Actual: [100, 200, 300]

Predicted: [110, 190, 280]

1. Mean Absolute Error (MAE)

MAE calculates the **average absolute difference** between actual and predicted values.

$$\text{MAE} = \sum |y - \hat{y}| / n$$

```
from sklearn.metrics import mean_absolute_error
```

```
mae = mean_absolute_error(y_test, y_pred)
```

```
print(mae)
```

Interpretation: If MAE = 20, the model is wrong by about **20 units on average**.

Advantage: Easy to understand and less affected by outliers than MSE.

2. Mean Squared Error (MSE)

MSE calculates the average of **squared errors**.

$$\text{MSE} = \sum (y - \hat{y})^2 / n$$

```
from sklearn.metrics import mean_squared_error
```

```
mse = mean_squared_error(y_test, y_pred)
```

```
print(mse)
```

Because errors are squared, **large errors are heavily penalized**.

3. Root Mean Squared Error (RMSE)

RMSE is simply the square root of MSE.

RMSE = $\sqrt{\text{MSE}}$

import numpy as np

from sklearn.metrics import mean_squared_error

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print(rmse)

Why use RMSE?

If you're predicting salary in ₹:

RMSE = ₹50,000

The error is expressed in the **same unit as the target**, making it easier to interpret.

4. R² Score

R² tells us **how much of the variation in the target variable is explained by the model**.

from sklearn.metrics import r2_score

r2 = r2_score(y_test, y_pred)

print(r2)

General interpretation:

R² = 1 → Perfect prediction

R² = 0 → No better than predicting the mean

R² < 0 → Worse than the mean baseline

For example:

R² = 0.80

means the model explains approximately **80% of the variance** in the target.

5. Adjusted R²

Normal R² can increase when you add more features, **even if those features are not useful**.

Adjusted R² accounts for the **number of features** and penalizes unnecessary ones.

Formula:

Adjusted R² =

$1 - [(1 - R^2)(n - 1) / (n - p - 1)]$

Where:

- $n \rightarrow$ number of observations
- $p \rightarrow$ number of features
- $R^2 \rightarrow R^2$ score

Python:

```
n = X_test.shape[0]
```

```
p = X_test.shape[1]
```

```
adjusted_r2 = 1 - (
```

```
(1 - r2) * (n - 1) / (n - p - 1)
```

```
)
```

```
print(adjusted_r2)
```

Multiple Linear Regression

Concept: Multiple Linear Regression is a **supervised learning algorithm** used to predict a **continuous numerical value** using **two or more input features**.

Simple vs Multiple Linear Regression

Simple Linear Regression:

1 feature $\rightarrow$ 1 target

CGPA $\rightarrow$ Package

Equation:

$$y = b_0 + b_1x_1$$

Multiple Linear Regression:

Multiple features $\rightarrow$ 1 target

CGPA + IQ + Experience $\rightarrow$ Package

Equation:

$$y = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + \dots + b_nx_n$$

Where:

- $y \rightarrow$ predicted output

- $b_0 \rightarrow$ intercept
- $b_1, b_2, \dots \rightarrow$ coefficients/weights
- $x_1, x_2, \dots \rightarrow$ input features

Geometric Intuition

With one feature, the model finds a **line**:

$$X \rightarrow y$$

With two features, it finds a **plane**:

$$X_1 + X_2 \rightarrow y$$

With more features, it becomes a **hyperplane**.

The goal is still the same: find the best-fitting relationship that minimizes prediction error.

Coefficients

Suppose the model learns:

$$\text{Package} = 1 + 2(\text{CGPA}) + 0.5(\text{IQ})$$

Here:

$$\text{Intercept} = 1$$

$$\text{CGPA coefficient} = 2$$

$$\text{IQ coefficient} = 0.5$$

A coefficient represents how the prediction changes when that feature increases by one unit, **holding the other features constant**.

Python Implementation

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

Multiple Linear Regression — Mathematical Formulation

MLR predicts y using multiple input features:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Matrix Form

The equation can be written as:

$$y = X\beta$$

- $X \rightarrow$ input/features matrix
- $\beta \rightarrow$ coefficient vector
- $y \rightarrow$ predicted output

Loss Function

The goal is to minimize the difference between actual and predicted values:

$$\text{Error} = \sum (y - \hat{y})^2$$

This is called **Sum of Squared Errors (SSE)**.

OLS Solution

By differentiating the loss function and setting the derivative to zero, we get:

$$\beta = (X^T X)^{-1} X^T y$$

This is the **Ordinary Least Squares (OLS)** solution.

Why Gradient Descent?

OLS requires calculating:

$$(X^T X)^{-1}$$

Matrix inversion becomes computationally expensive when there are **very large datasets or many features**.

So we can use **Gradient Descent**, which finds the optimal coefficients iteratively.

Key idea:

MLR $\rightarrow$ Minimize error $\rightarrow$ OLS can solve directly

$\rightarrow$ Gradient Descent for large problems

Linear Regression — 5 Assumptions

Linear Regression makes certain assumptions about the data to produce reliable results.

1. Linear Relationship

There should be an approximately **linear relationship between input features (X) and target (y)**.

Example:

Experience $\uparrow \rightarrow$ Salary $\uparrow$

Linear Regression works best when the relationship can be represented by a straight line.

2. No Multicollinearity

Input features should **not be highly correlated with each other**.

Example:

Age ↔ Experience

If two features contain almost the same information, it becomes difficult for the model to determine their individual effects.

Check: Correlation matrix or VIF.

3. Normal Distribution of Residuals

A **residual** is:

Residual = Actual – Predicted

Residuals should be approximately **normally distributed (bell-shaped)**.

This is mainly important for statistical inference such as confidence intervals and hypothesis testing.

4. Homoscedasticity

The **variance/spread of residuals should remain approximately constant** across different predicted values.

Constant error spread →  Homoscedasticity

Increasing/decreasing spread →  Heteroscedasticity

A funnel-shaped residual plot indicates heteroscedasticity.

5. No Autocorrelation

Residuals should be **independent of each other**.

Example of autocorrelation:

Today's error → related to yesterday's error

This is especially important in **time-series data**.

Check: Durbin-Watson test.
